INF-419 - Principles of Distributed Systems

Class Project 2026

First Deliverable

Ομάδα 6:

Βλάσσης Ιωάννης - 2021030139

Μπαλαντίνος Επιμενίδης - 2022030021

Φαρμάκης Αθανάσιος - 2022030003

GitHub Repo: https://github.com/taconoodle/MapReduce-Distributed-System-Implementation.git

Σκοπός:

Σκοπος αυτού του project είναι να υλοποιηθεί ένα κατανεμημένο σύστημα "χτίσμενο" πάνω σε Kubernetes. Το σύστημα έχει ως λειτουργικότητα την εκτέλεση του MapReduce αλγορίθμου, όπως αυτή προτάθηκε από τους Jeffrey Dean και Sanjay Ghemawat, μηχανικούς της Google, στις 6 Δεκεμβρίου του 2004. Λαμβάνει ως είσοδο από τους χρήστες ένα αρχείο JSON, μια μέθοδο map και μία μέθοδο reduce και με βάση αυτά παράγει το αποτέλεσμα σε ένα αρχείο JSON, το οποίο ο χρήστης μπορεί να κατεβάσει.

Περιγραφή Συστήματος:

Το σύστημα είναι σχεδιασμένο να τρέχει σε ένα ή σε πολλαπλά nodes. Σε κάθε node θα τρέχουν pods που ανήκουν σε ένα από τα παρακάτω services:

- **Authentication Service:** Το Authentication Service αποτελεί το σύστημα ταυτοποίησης του συστήματος. Είναι υπεύθυνο για την διαχείριση της ασφάλειας του συστήματος και την διαχείριση των χρηστών. Συγκεκριμένα, διατηρεί τα δεδομένα διαπίστευσης των χρηστών και εκδίδει access tokens δίνοντας στον χρήστη πρόσβαση στο σύστημα, ανάλογως με το επίπεδό του (admin/user). Το πρόγραμμα που είναι υπεύθυνο για την λειτουργία αυτή είναι το Keycloak (https://www.keycloak.org/) το οποίο διαφυλάσσει τους κωδικούς κρυπτογραφημένους και μέσω των προτύπων OAuth 2.0 και OpenID Connect, εκδίδει ένα JWT, το οποίο μπορεί το σύστημα να χρησιμοποιήσει για να επαληθεύσει την ταυτότητα του χρήστη, αλλά και τα δικαιώματα του, χωρίς να έχει πρόσβαση στον κωδικό του.

- **User Interface Service:** Το User Interface Service είναι υπεύθυνο για την αλληλεπίδραση του χρήστη με το σύστημα είτε μέσω GUI είτε μέσω CLI. Προωθεί τις εντολές του χρήστη στο backend ενώ ταυτόχρονα επικυρώνει τα access tokens (JWT). Ο χρήστης μπορεί να υποβάλει τα αρχεία δεδομένων στο UI τα οποία θα προωθήσει στο backend. Αντίστοιχα, το backend θα προωθήσει τα αποτελέσματα της δουλείας στο UI για να μπορέσει ο χρήστης να τα αποθηκεύσει. Ακόμη, εάν ένας χρήστης είναι ADMIN, το UI δίνει στον χρήστη πρόσβαση σε διαχειριστικές λειτουργίες, τη διαχείριση των χρηστών επικοινωνώντας με το Auth service αλλά και την διαχείριση των Nodes. Όσον αφορά τη διαχείριση των χρηστών, ο manager είναι υπεύθυνος για την προσθήκη, διαγραφή και επεξεργασία των δεδομένων των χρηστών στο σύστημα.
  Η UI υπηρεσία υλοποιείται ως Deployment καθώς τα δεδομένα δεν αποθηκεύονται μόνιμα εκεί και σε περίπτωση βλάβης το μόνο σημαντικό είναι η διεπαφή να είναι πάντα προσβάσιμη από τους χρήστες. Το UI είναι γραμμένο σε Python, χρησιμοποιεί FastAPI για την επικοινωνία του με τον Manager, το python-jose library για την διαχείριση των κλειδιών. Το GUI είναι υλοποιημένο σε HTML, CSS και JavaScript, ενώ το CLI (δεν έχουμε κατασταλάξει ακόμα), είναι γραμμένο σε Python, θα χρησιμοποιεί πιθανότατα το ttyd

πρόγραμμα για να δώσει στον χρήστη ένα virtual terminal στο browser του και είτε Click, είτε Typer για την υλοποίηση του CLI.

- **Distributed Data Service:** Το Disributed Data Service αποτελεί τη βάση δεδομένων του συστήματος. Είναι υπεύθυνο για να παρέχει συνεπή, αξιόπιστη και ανθεκτική σε σφάλματα αποθήκευση δεδομένων στο υπόλοιπο σύστημα.
  Η βάση αποθηκεύει πληροφορίες σχετικά με το κάθε job που εισάγει ο χρήστης (το job ID, το όνομα αρχείου εισόδου, το id του χρήστη, η ώρα εκκίνησης, το status της εργασίας κ.α.) καθώς και τα task από τα οποία αποτελείται το job (το path κάθε chunk στο SFS, το ID του worker που το αναλαμβάνει, το status του task κ.α.).
  Αποθηκεύει επίσης αυτόματα το keycloak ,στο schema του, τα δεδομένα που χρειάζεται για το authentication.
  Η DDS υπηρεσία υλοποιείται ως StatefulSet με 1 replica, ώστε να έχει την ίδια ταυτότητα και πρόσβαση στα ίδια δεδομένα σε περίπτωση που crashάρει το pod που την περιέχει.
  Είναι bound με ένα headless Kubernetes service, έτσι ώστε οι υπηρεσίες που προσπαθούν να επικοινωνήσουν μαζί της, να μπορούν να συνδεθούν απευθείας στο pod που φιλοξενεί την βάση δεδομένων.
  Το πρόγραμμα που θα χρησιμοποιήσουμε για αυτό το σκοπό είναι η PostgreSQL (https://www.postgresql.org/) .

- **Shared File System Service:** Το shared file system service αποτελεί τον χώρο αποθήκευσης του συστήματός μας. Έχει ως καθήκοντα την αποθήκευση των αρχείων που απαιτούνται για κάθε user job. Πιο συγκεκριμένα, διατηρεί τα Input Files που εισάγει ο χρήστης σε κάθε εργασία, τα chunks που προκύπτουν από τον διαχωρισμό του input αρχείου, τα ενδιάμεσα αρχεία που παράγονται μετά το map phase, τα αρχεία που παράγονται από το reduce phase και τα αρχείου εξόδου που προκύπτουν ως αποτέλεσμα του αλγορίθμου και θα λάβει τελικά ο χρήστης.
  Το SFS Service είναι υλοποιημένο ως StatefulSet με 3 replicas, bound με ένα headless service το οποίο επιτρέπει την απευθείας σύνδεση σε κάθε replica. Η επιλογή του replica που θα χρησιμοποιηθεί για την αποθήκευση αρχείων του κάθε user job επιλέγεται με βάση την εξής συνάρτηση:

  $$hash(job_id) \% num_of_sfs_replica$$

  Το πρόγραμμα που επιλέξαμε για να διεκπεραιώσει το ρόλο του SFS service είναι το RustFS (https://rustfs.com/) που χρησιμοποιεί το API των S3 Storage Systems των Amazon Web Services.

- **Manager Service:** Το manager service αποτελεί τον εγκέφαλο του συστήματος. Είναι υπεύθυνο για την διαχείριση των jobs που εισάγει ο χρήστης.
  Πριν από κάθε ενέργεια ο manager κάνει validate τα access tokens διασφαλίζοντας έτσι την ασφάλεια.

Για την διαχείριση των εργασιών από την άλλη, είναι υπεύθυνος για την εκκίνηση μιας εργασίας, την παρακολούθησή της όσο αυτή τρέχει και τη διαχείριση της ολοκλήρωσης και προώθηση του αποτελέσματος (ειδοποίηση του χρήστη σχετικά με την επιτυχία ή αποτυχία της).
Το manager service είναι υλοποιημένο ως StatefulSet με 3 replicas, bound με ένα headless service που επιτρέπει την απευθείας σύνδεση σε κάθε replica. Το replica που είναι υπεύθυνο για την επεξεργασία των αιτημάτων κάθε user θα επιλέγεται με βάση τη συνάρτηση:

hash(user_id) % num_of_manager_replicas

Για παράδειγμα όταν ένας χρήστης με user_id == 3 επιχειρήσει να υποβάλει ένα νέο job, το αίτημα του θα προωθηθεί στον manager-0. Αν παράλληλα ο χρήστης με user_id == 10 θελήσει να ακυρώσει κάποιο job που έχει υποβάλει, τότε το αίτημά του θα προωθηθεί στον manager-1 που θα το αναλάβει, οι διεργασίες του user_id == 11 που τρέχουν στο background παρακολουθούνται και διαχειρίζονται από τον manager-2.
Το manager είναι γραμμένο σε Python και χρησιμοποιεί το psycopg2 library για την επικοινωνία του με την DDS, το boto3 API για την επικοινωνία του με το SFS, το FastAPI για την επικοινωνία του με το UI, τους Worker και το Authentication Service και το filesplit library για τον διαχωρισμό των αρχείων εισόδου σε chunks. Χρησιμοποιεί επίσης το kubernetes API για την επικοινωνία και διαχείριση με το cluster

- **Workers:** Οι worker διακρίνονται σε δύο κατηγορίες. Στους Map Worker και Reduce Worker, αναλόγως τη διεργασία που καλούνται να εκτελέσουν.
  - Οι Map Worker αναλαμβάνουν να εκτελέσουν το Map κομμάτι του MapReduce αλγορίθμου σε ένα chunk του input αρχείου του χρήστη. Γνωρίζοντας το path του chunk στο Shared File System κατά την εκκίνηση τους, εφαρμόζουν σε αυτό την map μέθοδο που έχει προμηθεύσει ο χρήστης στο σύστημα και αποθηκεύουν το αποτέλεσμα τους σε ένα ξεχωριστό bucket στο SFS.
  - Οι Shuffle Worker αναλαμβάνουν να ταξινομήσουν και να ενώσουν τα αρχεία που προκύπτουν από τους Map Workers για να χρησιμοποιηθούν από τους Reduce Worker. Κάθε Shuffle Worker είναι υπεύθυνος για να τακτοποιήσει τα αρχεία ενός Reduce Worker και θα εξασφαλίσει οτί αυτός θα λάβει ένα ενωποιημένο, ταξινομημένο αρχείο δεδομένων.
  - Οι Reduce Workers αναλαμβάνουν την εκτέλεση του Reduce κομματιού του MapReduce αλγορίθμου. Με τη σειρά τους αυτοί φροντίζουν να εφαρμόσουν την reduce μέθοδο που έχει προμηθεύσει ο χρήστης στα ενδιάμεσα αρχεία που προέκυψαν από τους Map Workers. Τοποθετούν το αποτέλεσμά τους σε δικό τους ξεχωριστό bucket.

Οι workers υλοποιούνται ως Kubernetes Jobs. Δημιουργούνται από τον Manager στην αρχή κάθε Map ή Reduce φάσης και το Kubernetes φροντίζει να τους “ξανασηκώσει” σε

περίπτωση που crashάρουν. Αν crashάρουν πολλές φορές τότε το task τους δηλώνεται ως αποτυχημένο και ο Manager φροντίζει να το διαχειριστεί καταλλήλως. Είναι επίσης γραμμένοι σε Python.

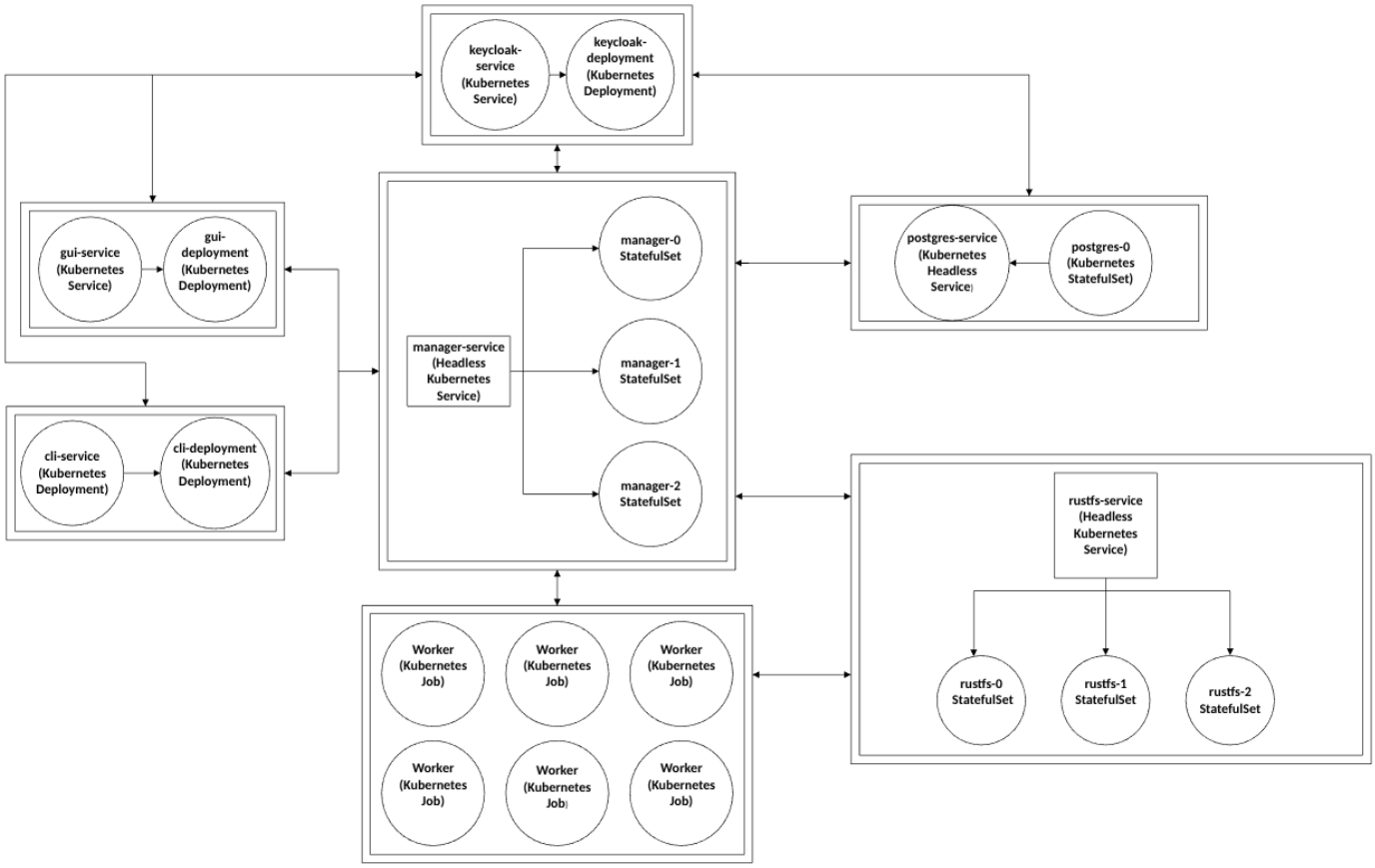
Όπου χρειαζόμαστε αποστολή HTTP requests έχουμε χρησιμοποίησει το requests library στα πρότυπά μας. Ωστόσο, πρόσφατα είδαμε πως υπάρχει το HTTPX, το οποίο θεωρείται πως λειτουργεί καλύτερα με το FastAPI, οπότε θα εξεταστεί αν θα χρησιμοποιηθεί αυτό στη θέση του.

Σε κάθε περίπτωση, η τελική παρουσίαση θα αναφέρει τα σωστά libraries όπου αυτά χρησιμοποιούνται.

Κατά την υλοποιήση αυτού του project χρησιμοποιούμε το Google Gemini ως coding agent για την ταχύτερη παραγωγή και κατανόηση του κώδικα.

Δομή Συστήματος

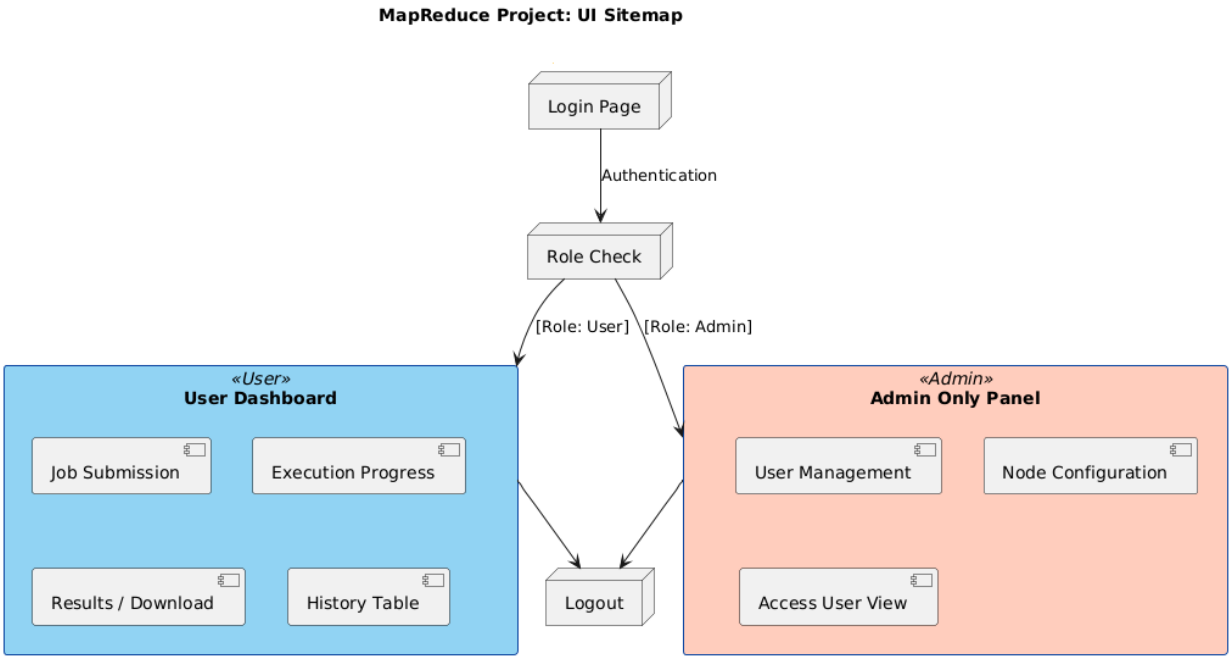
Παρακάτω παρουσιάζουμε τη δομή του συστήματός μας σε διάγραμμα, στο οποίο φαίνεται το είδος του κάθε pod, τα replicas, όπου αυτά έχουν προκαθορισμένο πλήθος καθώς και ποια services έχουν μεταξύ τους συνδέσεις.



UML διαγράμματα

Παρουσιάζουμε ενδεικτικά με τη χρήση UML activity διαγραμμάτων μερικές από τις βασικές λειτουργίες του συστήματος μας, συμπληρώνοντας πληροφορίες όπου αυτό χρειάζεται:

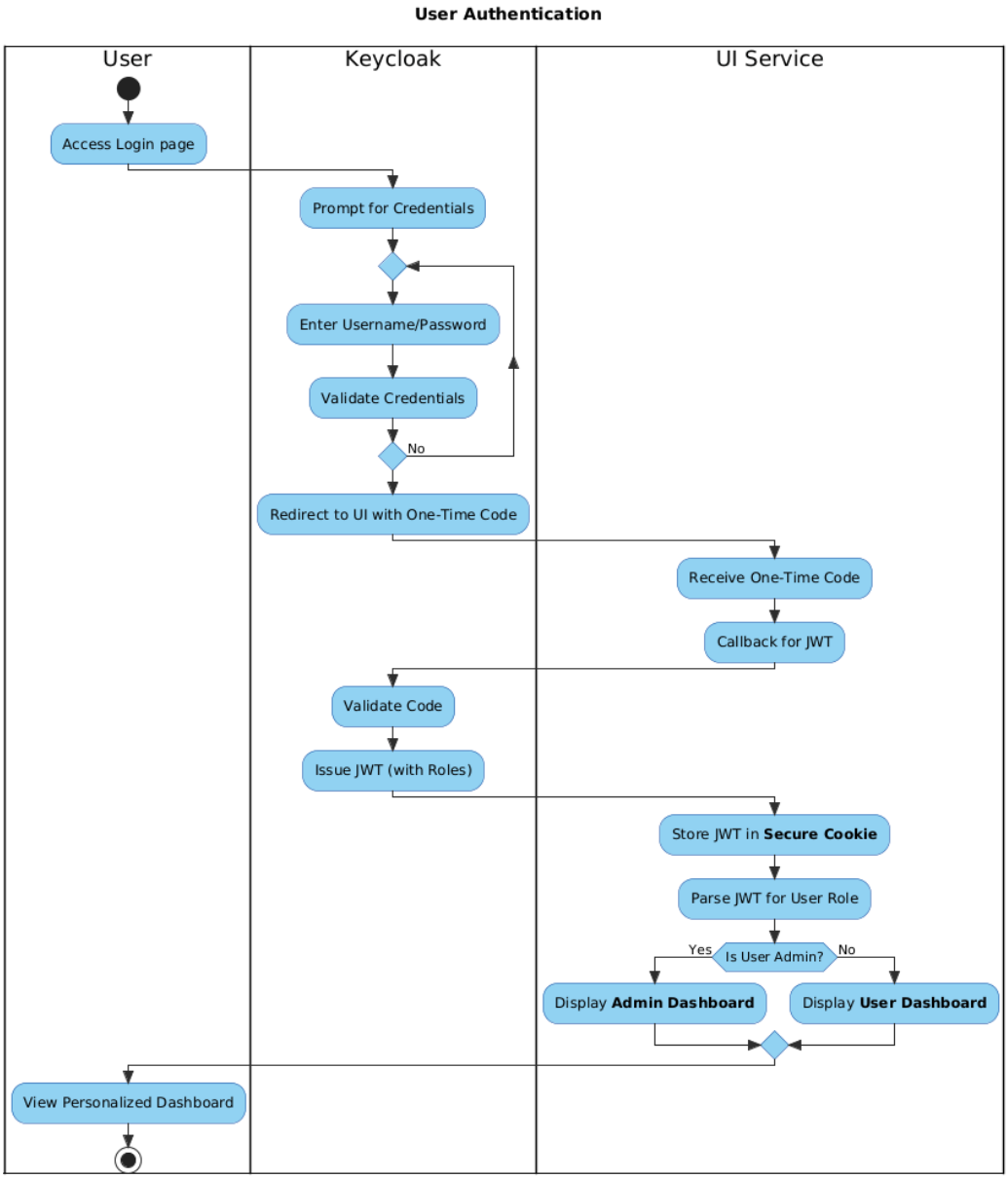
– **UI Sitemap**



Το UI αποτελείται από το login Page που κάνει redirect τον χρήστη για να κάνει authenticate. Από εκεί, η εφαρμογή χωρίζεται σε δύο βασικά pages ανάλογα με τον ρόλο του χρήστη:

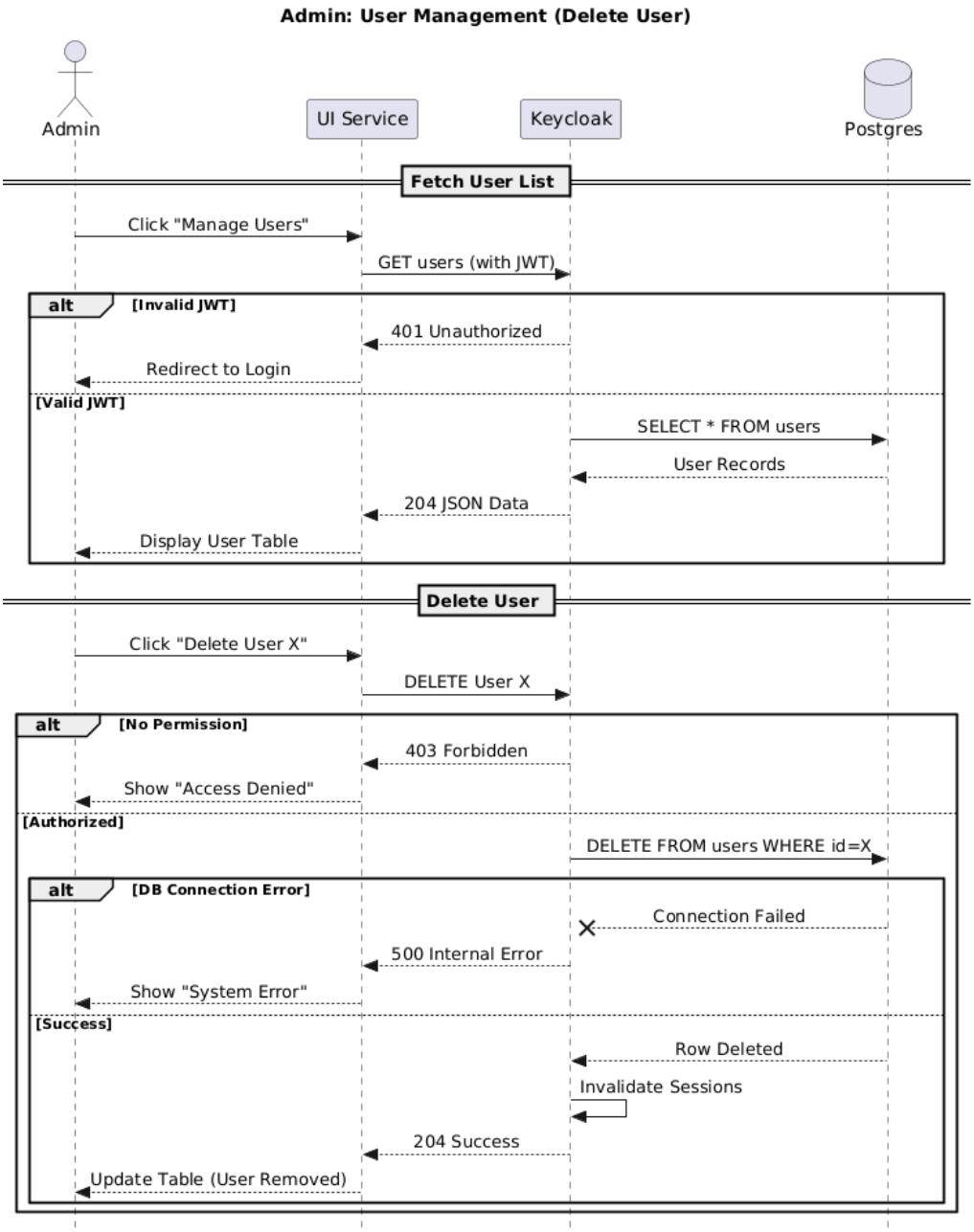
- User: Έχει πρόσβαση στις λειτουργίες υποβολής εργασιών, παρακολούθησης των εργασιών αλλά και λήψης αποτελεσμάτων.
- Admin: Έχει πρόσβαση στις λειτουργίες του χρήστη αλλά και σε διαχειριστικές λειτουργίες, τη διαχείριση των χρηστών επικοινωνώντας με το Auth service αλλά και την διαχείριση των Nodes.

– **User Authentication**



Για την πρόσβαση στις λειτουργίες του συστήματος, ο χρήστης πρέπει να κάνει Login μέσω της υπηρεσίας Keycloak. Το Keycloak λειτουργεί ως υπεύθυνος για την αυθεντικοποίηση των χρηστών. Μετά την επιτυχή ταυτοποίηση το Keycloak εκδίδει ένα one time code στέλνοντας στο UI για να κάνει callback ώστε να δημιουργηθεί ασφαλής server-to-server σύνδεση. Η διαδικασία αυτή ολοκληρώνεται με το Keycloak να εκδίδει ένα JWT. Το Token αποθηκεύεται από τον browser με την χρήση ενός Secure Cookie. Τελικά, το UI ελέγχει τα δικαιώματα του χρήστη και τελικά εμφανίζει το ανάλογο dashboard στον χρήστη.

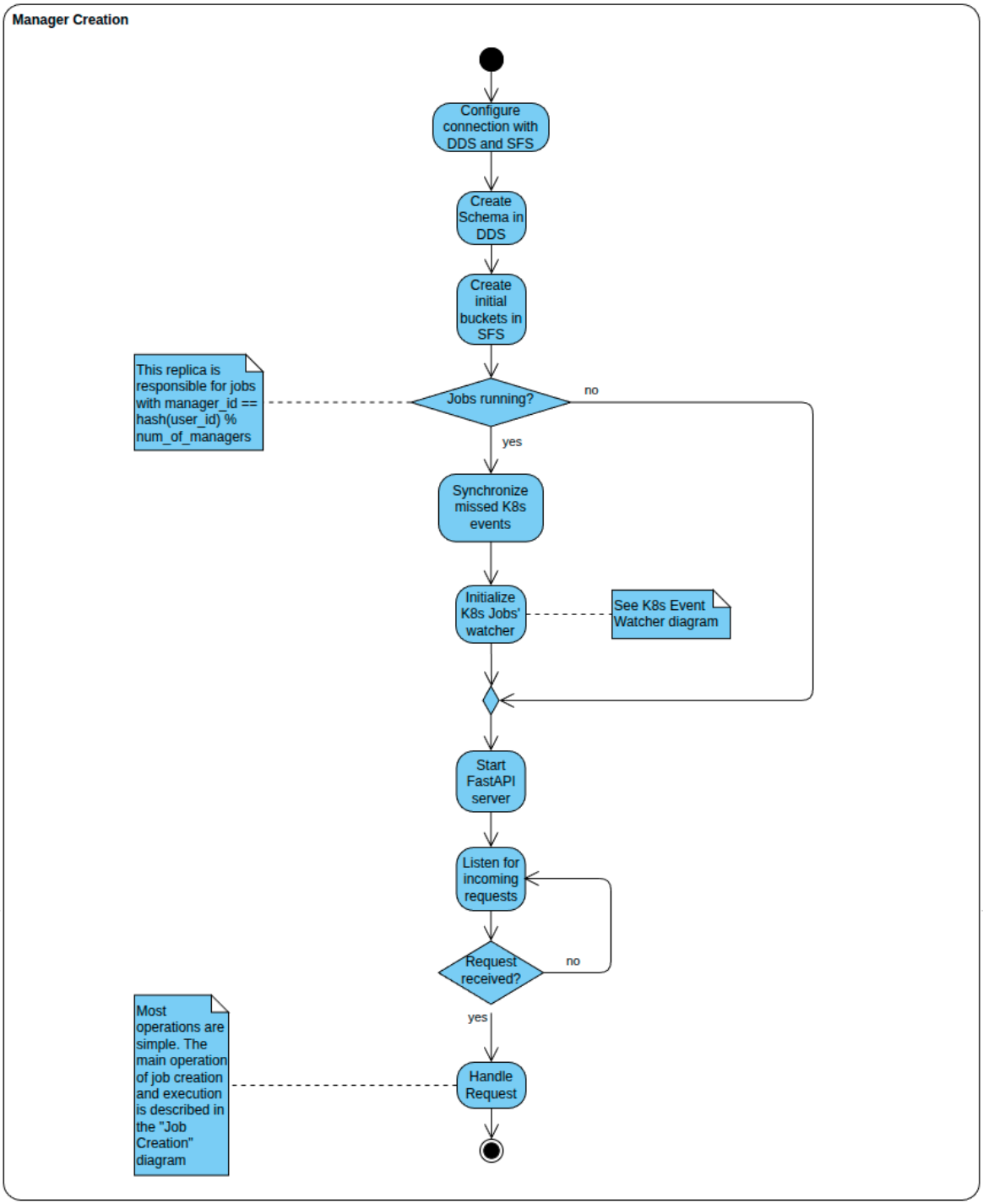
– **User Management (Delete Scenario)**



Όταν ένας logged in Admin επιθυμεί να διαγράψει έναν User επιλέγει Manager Users. Το UI Service χρησιμοποιώντας το JWT στέλνει ένα αίτημα διαγραφής στο Keycloak. Το Keycloak ελέγχει εάν το JWT είναι valid admin token και εάν δεν έγκυρο, το Keycloak απορρίπτει το αίτημα επιστρέφοντας error 403 forbidden και το UI κάνει redirect για login. Εάν είναι έγκυρο το το keycloak επικοινωνεί με τη βάση δεδομένων και εκτελεί transaction επιστρέφοντας όλους τους users στο Keycloak που μετατρέπει τα δεδομένα σε JSON και τα στέλνει πίσω στο UI. Τελικά, ο χρήστης βλέπει τον πίνακα με τους χρήστες και μπορεί να

επιλέξει ποιον επιθυμεί να διαγράψει. Αφού ο Admin επιλέξει τον συγκεκριμένο χρήστη προς διαγραφή από τον πίνακα, το UI στέλνει DELETE Request στο Keycloak και ακολουθείται η ίδια διαδικασία. Ακόμη, όταν διαγράφουμε τον χρήστη δεν αρκεί να αφαιρέσουμε τα δεδομένα του από τη βάση αλλά χρειάζεται και να τον αποσυνδέσουμε από το σύστημα εάν είναι συνδεδεμένος . Φυσικά, εάν υπάρχει failure στην επικοινωνία με το database, το keycloak επιστρέφει ένα error 500 στο UI Service. Σε αυτή την περίπτωση, το transaction ακυρώνεται, ο χρήστης δεν διαγράφεται και ο Admin βλέπει ένα μήνυμα σφάλματος.

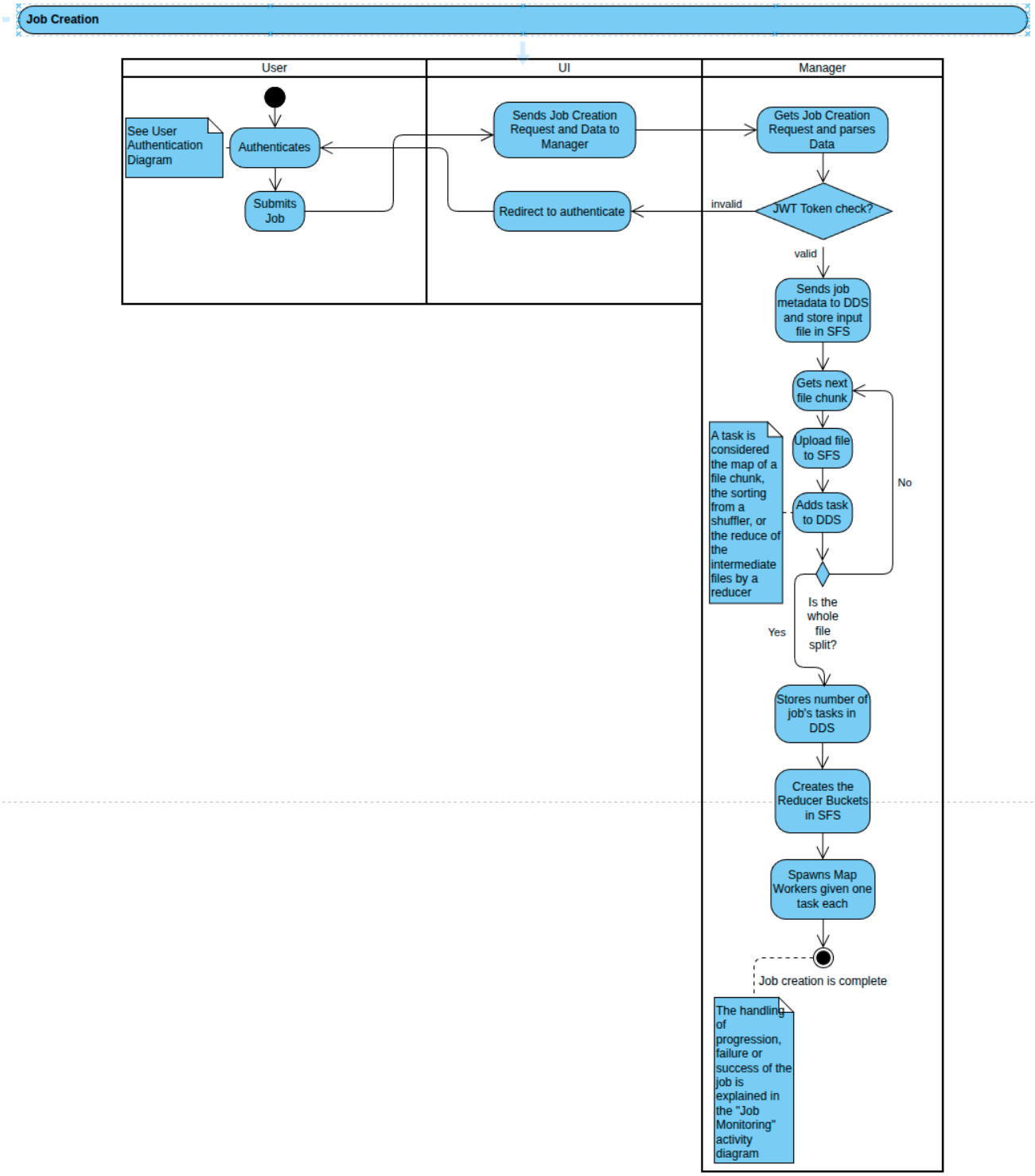
- **Δημιουργία/εκκίνηση/επανεκκίνηση Manager Replica:**



Κατά την εκκίνηση του Manager, χρειάζεται να κάνει configure τις συνδέσεις του με τις υπηρεσίες που απαιτούν σύνδεση με API για την επικοινωνία, δηλαδή με το DDS και το SFS καθώς και να ελέγξει αν αυτές είναι ρυθμισμένες και αρχικοποιημένες (π.χ. αν τα tables στην βάση δεδομένων

είναι δημιουργημένα και έτοιμα να δεχτούν δεδομένα), σε περίπτωση που είναι η πρώτη εκκίνηση του manager service. Έπειτα, πρέπει να καλυφθεί η περίπτωση που ο manager "ξανασηκώνεται" μετά από προηγούμενο crash του, οπότε είναι πιθανό, αν του είχαν ανατεθεί εργασίες να έχει χάσει συμβάντα από τους worker, για τα οποία θα πρέπει να ενημερωθεί με τη χρήση του K8s' API. Όταν ελέγξει και διαχειριστεί τα τυχόν χαμένα συμβάντα, εκκινεί τον watcher συμβάντων ώστε να μπορεί να αντιδρά σε επιτυχίες ή αποτυχίες των worker. Περισσότερα για το monitoring αυτό περιγράφονται παρακάτω. Έπειτα, δημιουργεί instance του FastAPI και αναμένει να δεχτεί requests από το UI. Ενδεικτικά παρουσιάζεται η διαχείριση ενός Job Creation request παρακάτω.

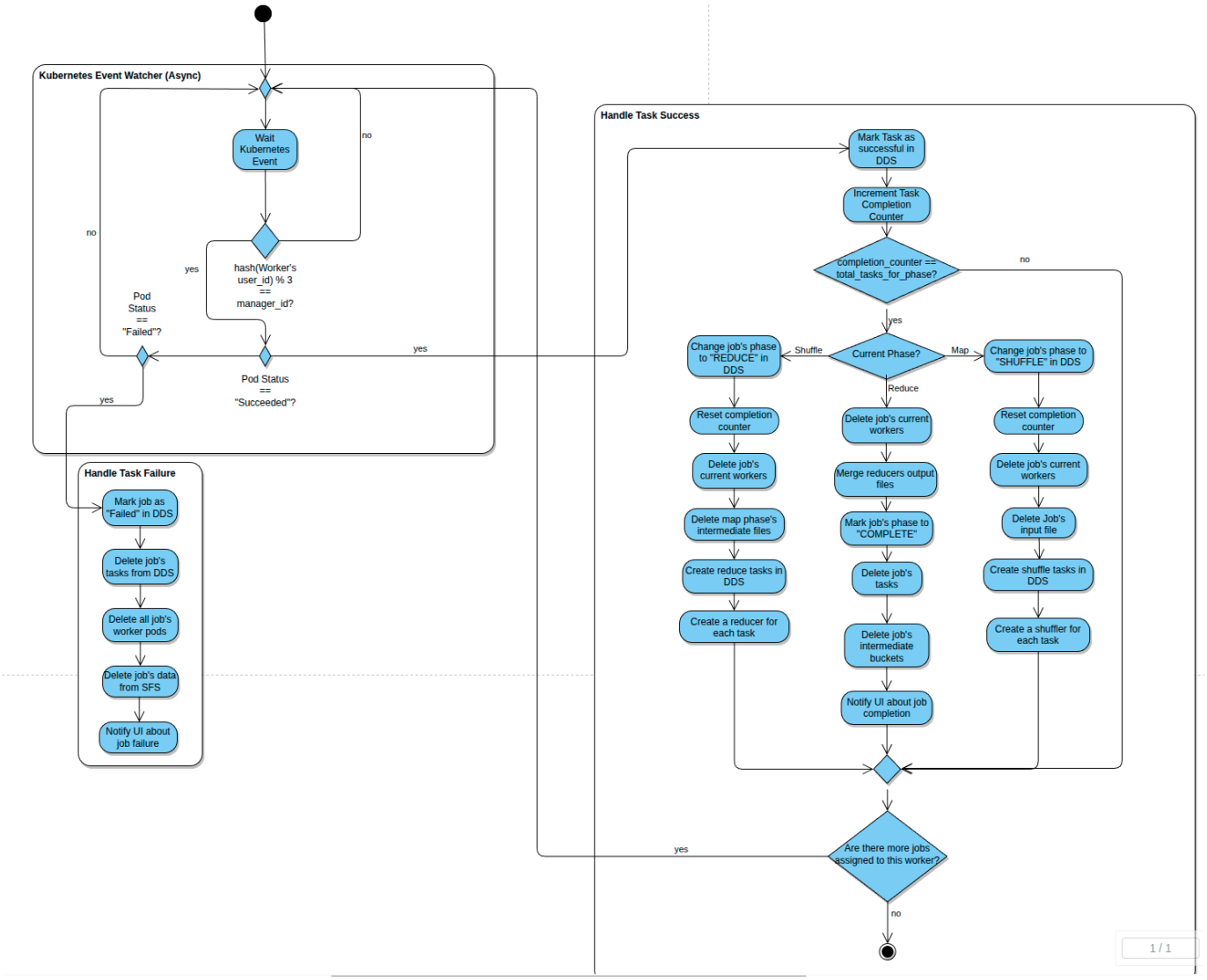
- **Job Creation**



Όπως βλέπουμε, κατά τη δημιουργία ενός job χρήστη, πρέπει πρώτα ο ίδιος ο χρήστης να κάνει login ή register με τη χρήση του UI και έπειτα, θα προωθηθεί ένα job creation request στο manager service που θα το αναλάβει. Αν το JWT token του χρήστη είναι έγκυρο, θα πρέπει να καταγραφεί το

job στη βάση δεδομένων, να γίνει upload το αρχείο του χρήστη στο SFS και να αρχίσει το “σπάσιμο” του input αρχείου στο χρήστη σε chunks των 64MB. Κάθε chunk που ετοιμάζεται γίνεται upload στο SFS και δημιουργείται ένα MAP task γι’ αυτό στην βάση. Όταν όλο το αρχείο γίνει split, στο entry του job στην βάση προστίθεται και ο συνολικός αριθμός chunk που θα πρέπει να επεξεργαστούν, δηλαδή ο αριθμός task που θα πρέπει να ολοκληρωθεί για να θεωρηθεί το Map phase ολοκληρωμένο. Έπειτα δημιουργούνται τα απαραίτητα buckets για την εκτέλεση της εργασίας και εκκινούνται οι worker που θα αναλάβουν to map phase.

- **Worker Event Watcher**



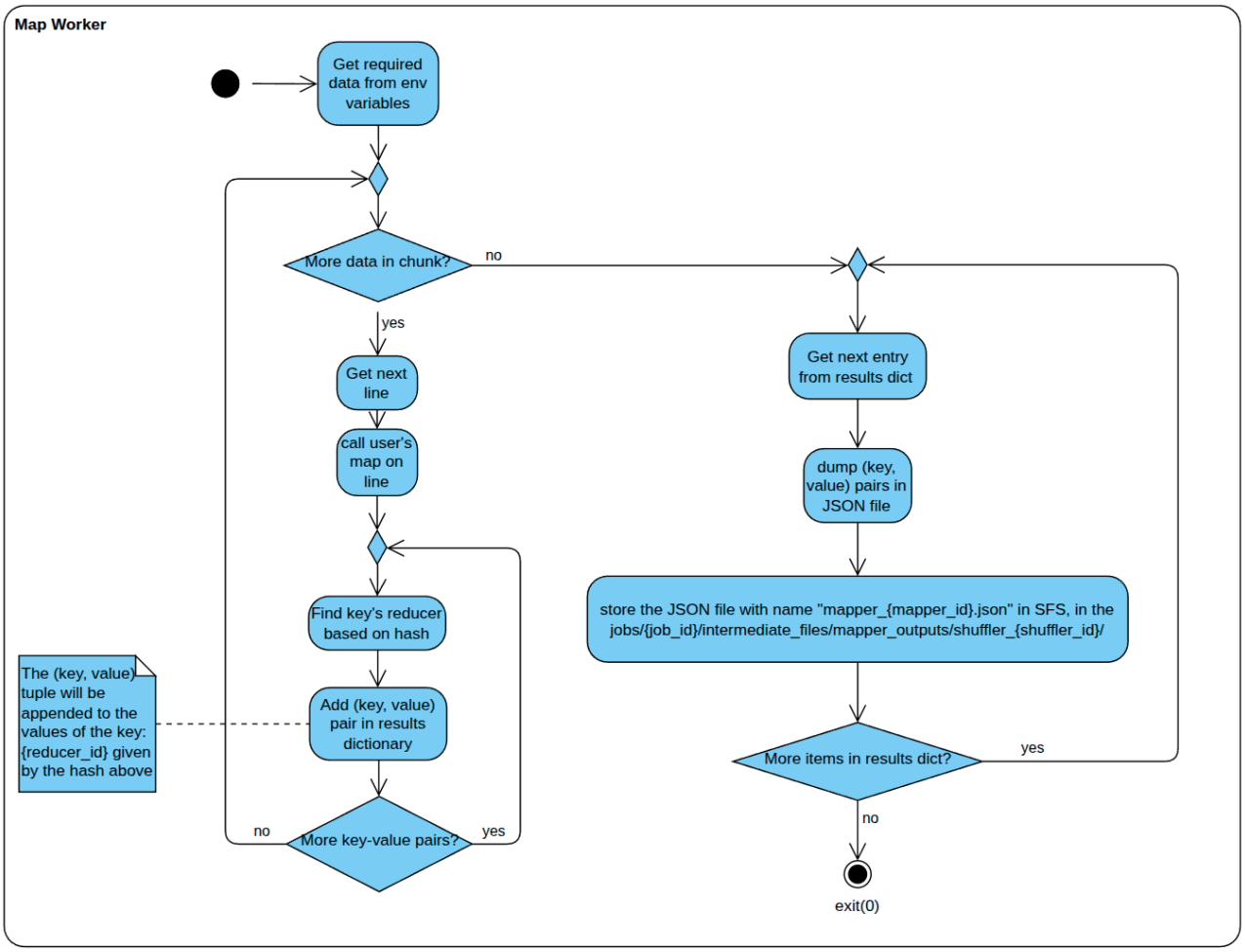
Στο background ενός manager service, εφόσον υπάρχουν running εργασίες που αναλογούν στο συγκεκριμένο manager replica τρέχει ένα Kubernetes event watcher, το οποίο κάνει monitor και εντοπίζει την αλλαγή κάποιου worker σε κατάσταση “Failed” ή “Succeeded”.

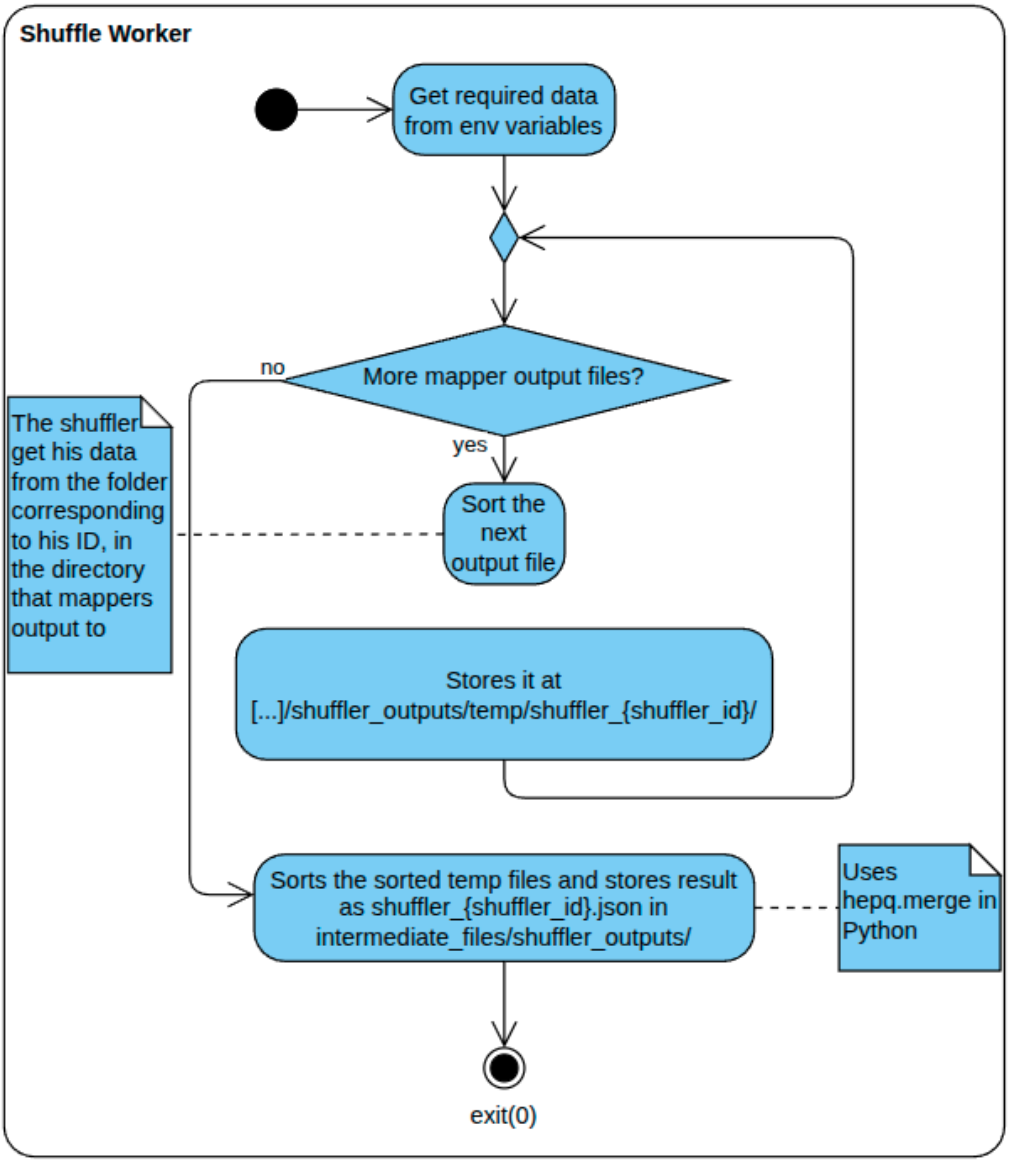
Αν το watcher εντοπίσει πως κάποιο worker έχει αποτύχει, τότε θεωρεί το task του ανεπιτυχές και μαρκάρει ολόκληρο το job ως ανεπιτυχές, φροντίζοντας να καθαρίσει τα αρχεία του και να ειδοποιήσει το UI σχετικά.

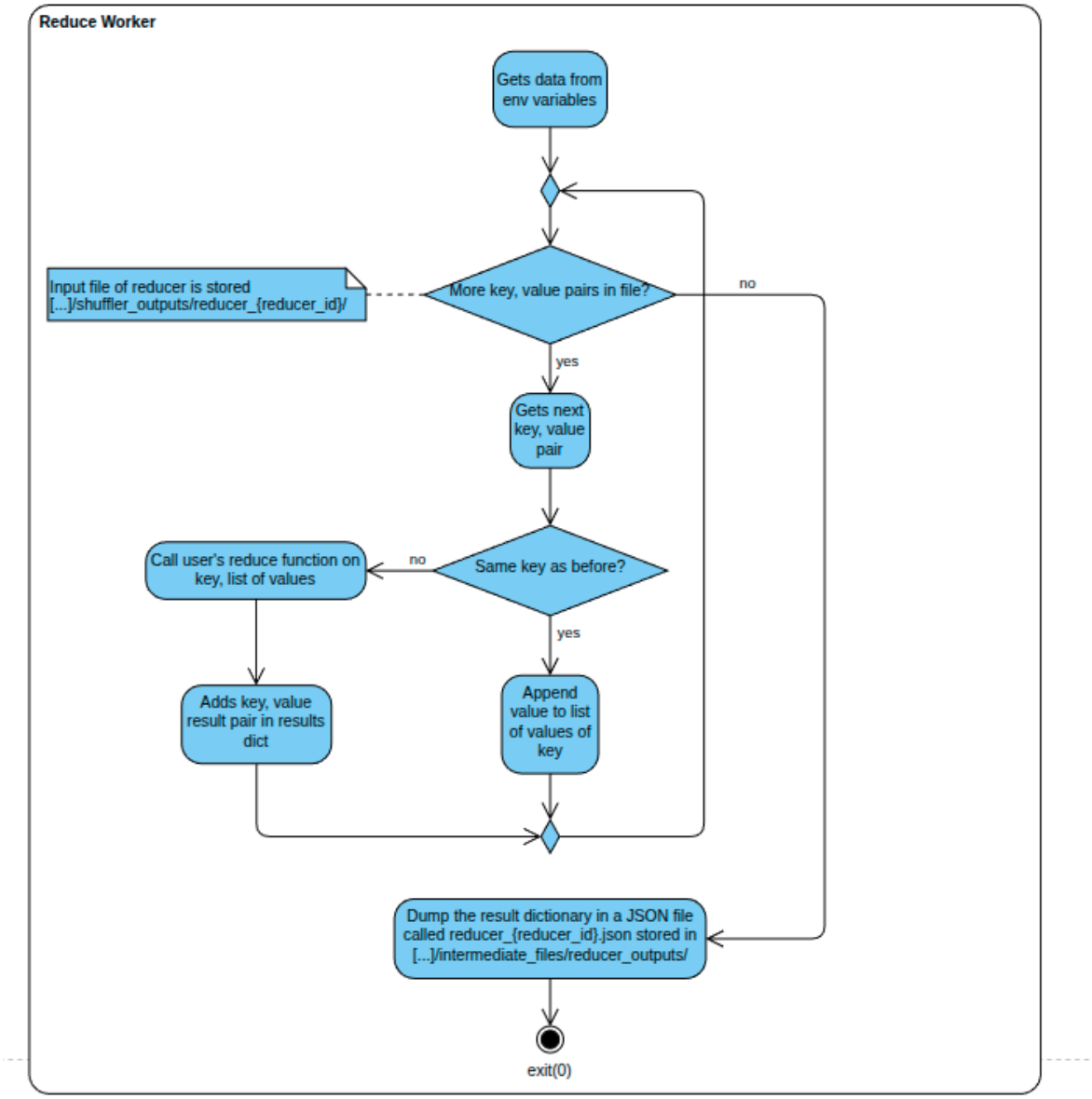
Αν το watcher εντοπίσει πως το worker είναι “Succeeded”, για αρχή μαρκάρει το task που είχε ο watcher ως επιτυχές και κάνει increment τον counter επιτυχημένων task. Αν έπειτα, το counter επιτυχημένων task γίνει ίσο με τον απαιτούμενο αριθμό (αριθμός job chunks για map phase, αριθμός reducer για shuffle και reduce), φροντίζει να καθαρίσει προηγούμενα άχρηστα αρχεία, να δημιουργήσει νέα task και να κάνει deploy νέους worker για την επόμενη φάση. Όταν πάλι

τελειώσει και το Reduce phase, κάνει merge τα αρχεία output των reducer, ενημερώνει τη βάση για την ολοκλήρωση της εργασίας, καθαρίζει τα αρχεία της επεξεργασίας, διαγράφει τα task του job και ενημερώνει το UI σχετικά με την ολοκλήρωση.

- **Workers**


Map Worker
Get required data from env variables
More data in chunk?
no
yes
Get next line
call user's map on line
Find key's reducer based on hash
Add (key, value) pair in results dictionary
The (key, value) tuple will be appended to the values of the key: {reducer_id} given by the hash above
More key-value pairs?
no
yes
Get next entry from results dict
dump (key, value) pairs in JSON file
store the JSON file with name "mapper_{mapper_id}.json" in SFS, in the jobs/{job_id}/intermediate_files/mapper_outputs/shuffler_{shuffler_id}/
More items in results dict?
yes
no
exit(0)


Shuffle Worker
Get required data from env variables
More mapper output files?
no
yes
Sort the next output file
The shuffler get his data from the folder corresponding to his ID, in the directory that mappers output to
Stores it at [...]/shuffler_outputs/temp/shuffler_{shuffler_id}/
Sorts the sorted temp files and stores result as shuffler_{shuffler_id}.json in intermediate_files/shuffler_outputs/
Uses hepq.merge in Python
exit(0)



Οι Workers διακρίνονται σε τρία είδη:

- **Map Worker:** Ο Map Worker αφού λάβει το metadata του task του μέσω των environment variables που του δίνονται από τον manager κατά τη δημιουργία του, ξεκινάει να εφαρμόζει το map function του χρήστη στο chunk που του έχει ανατεθεί προσθέτοντας κάθε key, value pair σε ένα tuple, το οποίο γίνεται append σε μία λίστα. Η λίστα αυτή είναι το value του αριθμού του reducer που αντιστοιχεί στο key σε ένα results dictionary. Ο αριθμός αυτός προκύπτει από το hash του κλειδιού, όπως έχει αναφερθεί παραπάνω. Όταν τελειώσει η επεξεργασία του chunk τα αποτελέσματα εισάγονται σε αρχεία JSON τα οποία αποθηκεύονται στα σωστά path ώστε να βρεθούν από τους shuffler.

- **Shuffle Worker:** Ο Shuffle Worker, λαμβάνει ομοίως το metadata του task του μέσω των environment variables που δίνονται από τον manager. Έπειτα στο path που του αναλογεί ταξινομεί το αρχείο που έχει προκύψει από τον κάθε map worker και, όταν τα ταξινομήσει όλα, χρησιμοποιεί τη μέθοδο heapq.merge για να ταξινομήσει τα περιεχόμενα όλων των αρχείων σε ένα, κοινό αρχείο, το οποίο και θα χρησιμοποιήσει ως είσοδο ένας reduce worker.
- **Reduce Worker:** Ο Reduce Worker και πάλι λαμβάνει το metadata του task του μέσω των environment variables και αντλεί τα δεδομένα του όπως ειπώθηκε από το αρχείο εξόδου του αντίστοιχου shuffle worker στο προηγούμενο στάδιο του job. Ο reduce worker θα ξεκινήσει και για κάθε κλειδί στο αρχείο εισόδου του θα καλέσει το reduce function του χρήστη με ορίσματα το κλειδί και τη λίστα με τις τιμές αυτού του κλειδιού. Έπειτα, το αποτέλεσμα της συνάρτησης θα το κάνει append σε ένα κοινό dictionary, το οποίο και στο τέλος της εκτέλεσης του worker θα τοποθετηθεί στο path αποτελεσμάτων, ώστε να χρησιμοποιηθεί από τον manager για το τελικό merge.

<u>Υποσημείωση</u>: Σκεφτόμαστε να υλοποιήσουμε τη λειτουργία που περιγράφεται στο αρχικό paper του MapReduce αλγορίθμου, αλλά δεν έχουμε καταφέρει να σκεφτούμε ακριβώς την υλοποίησή του ώστε να την ενσωματώσουμε στα διαγράμματα. Η βασική ιδέα είναι όταν μείνουν λίγοι worker σε κάθε phase ή όταν παρουσιαστεί κάποιος αργός worker, να δημιουργούνται νέοι worker που θα αναλάβουν παράλληλα με τους αρχικούς τα task που δεν έχουν ολοκληρωθεί. Όποιος worker τελειώσει πρώτος θεωρείται επιτυχής και κρατάμε το αποτέλεσμά του. Ο άλλος διαγράφεται και τα αποτελέσματά του απορρίπτονται. Ωστόσο, αυτό μας δημιουργεί θέματα με τα path που θα πρέπει να θέσουμε και με τον τρόπο των output των worker, οπότε θα χρειαστεί να σκεφτούμε περαιτέρω την υλοποίηση.

Το άλλο πρόβλημα που δεν έχουμε προλάβει να σκεφτούμε είναι το γεγονός πως ο κάθε reducer μπορεί να έχει ιδιαίτερα μεγάλο όγκο πληροφορίας να αναλύσει. Μια σκέψη που έχουμε είναι να αποθηκεύει στο SFS μέρος της εργασίας του κάθε x byte. Αυτό όμως μας οδηγεί ακριβώς στο πρόβλημα με τα path που αναφέραμε στην πάνω παράγραφο. Αυτός είναι και ο λόγος που διαχωρίσαμε και προσθέσαμε τους shuffle worker. Στην αρχή σκοπεύαμε να έχουμε το sort ως μέρος του reduce phase, αλλά ο όγκος των δεδομένων θα ήταν πρόβλημα για μια ακριβή διεργασία όπως η ταξινόμηση, οπότε σε περίπτωση, είτε backup reducer, είτε κάποιου reducer που επανεκκινείται, θα αυξανόταν ο φόρτος εργασίας.

Σε κάθε περίπτωση τα τελικά διαγράμματα θα ακολουθούν την τελική υλοποίηση του συστήματος και οι περιγραφές θα ενημερωθούν.

Δομή Βάσης

Η βάση μας περιλαμβάνει δύο πίνακες, το πίνακα με τα job και το πίνακα με τα tasks. Ο πίνακας task είναι weak entity του πίνακα job δηλαδή χωρίς το πίνακα job δεν μπορεί να υπάρχει ο πίνακας task. Επιπλέον, η πληθικότητα αυτών των δύο πινάκων είναι 1:N δηλαδή ένα job μπορεί να περιλαμβάνει 1-N tasks ενώ κάθε task περιλαμβάνεται σε ένα μόνο job. Τέλος, το task έχει ολική συμμετοχή στη σχέση με το job, καθώς δεν υφίσταται να υπάρχει κάποιο task που να μην ανήκει σε κάποιο συγκέκριμένο job.

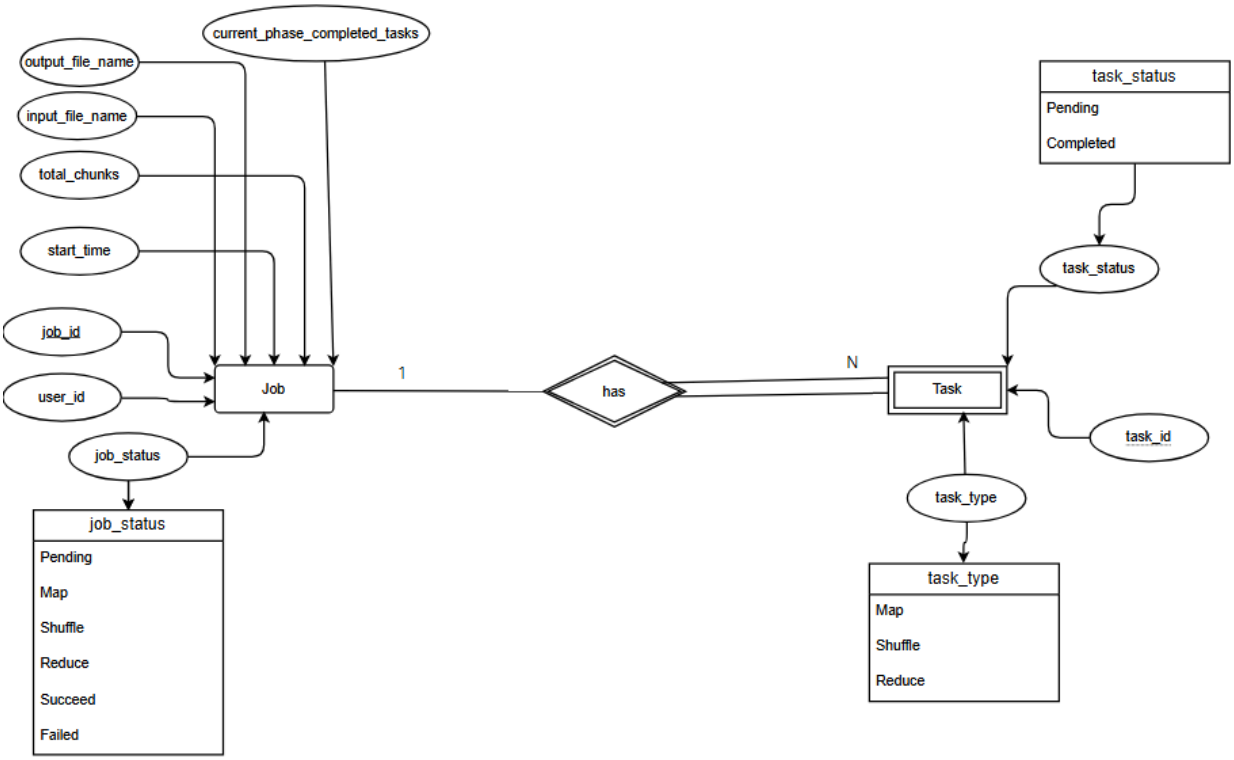
Πίνακας Job

Ο πίνακας Job περιλαμβάνει το job_id που είναι τύπου uuid και αποτελεί το Primary key του πίνακα. Επιπλέον, στο συγκεκριμένο πίνακα θα υπάρχουν και οι στήλες user_id τύπου uuid και start_time(τύπου time) που θα δηλώνει την ώρα έναρξης του συγκεκριμένου job. Ακόμη, υπάρχουν οι στήλες input_file_name και output_file_name (τύπου Varchar και οι δύο) και δηλώνουν το path, για να γνωρίζει το σύστημα από που να αντλήσει το αρχείο εισόδου και πως να ονομάσει το αρχείο εξόδου. Υπάρχει και το job status τύπου enum που περιλαμβάνει τις τιμές Pending, Map, Shuffle, Reduce, Succeed και Failed. Τέλος, υπάρχει η στήλη total_chunks τύπου integer που εκφράζει τον συνολικό αριθμό των chunk αρχείων που προέκυψαν κατά τη φάση του split, καθω και η στήλη current_phase_completed_tasks που είναι τύπου integer που εκφράζει τον αριθμό των ολοκληρωμένων tasks σε κάθε φάση και ενημερώνεται από το manager κάθε φορά που ολοκληρώνεται κάποιο task κάποιας φάσης. Συγκεκριμένα, η φάση map του συγκεκριμένου job γίνεται complete όταν η μεταβλητή αυτή ισούται με το πλήθος των chunks που προήλθαν από το split, ενώ τα στάδια shuffle και reduce, θεωρούνται complete όταν η μεταβλητή current_phase_completed_tasks ισούται με τον αριθμό των reducer worker που θα υπάρχουν για αυτό το job.

Πίνακας Task

Ο πίνακας task περιλαμβάνει το task_id που είναι serial number και αποτελεί το μερικό κλειδί του συγκεκριμένου πίνακα. Το primary key αυτού του πίνακα αποτελούν το ζεύγος(job_id, task_id). Επιπλέον, ο πίνακας περιλαμβάνει το task_type που είναι enum και παίρει τιμές από το παρακάτω σύνολο: Map, Shuffle, Reduce και υπάρχει και το task_status που είναι enum και παίρνει τη τιμή είτε Pending είτε Completed.

Παρακάτω δίνεται και το σχετικό schema.



Δομή Shared File System

Παρουσιάζουμε παρακάτω σε μορφή δέντρου την δομή που θα έχει το shared file system και αναλύω συνοπτικά τη χρήση κάθε φακέλου.

```
.
`-- RustFS/
    `-- jobs/
        `-- {job_id}/
            |-- input_files/
            |   `-- job_{job_id}.json
            |-- output_files/
            |   `-- job_{job_id}.json
            `-- intermediate_files/
                |-- chunks
                |-- mapper_outputs/
                |   `-- shuffler_{shuffler_id}/
                |       `-- mapper_{mapper_id}.json
                |-- shuffler_outputs/
                |   |-- reducer_{reducer_id}/
                |   |   `-- shuffler_{shuffler_id}.json
                |   `-- temp/
                |       `-- shuffler_{shuffler_id}/
                |           `-- sorted_mapper_{mapper_id}.json
                `-- reducer_outputs/
                    `-- reducer_{reducer_id}.json
```

Δίνουμε και ένα link αν προτιμάται:

SFS Structure

Όπως βλέπουμε μέσα στο SFS υπάρχει το jobs bucket που είναι κατά έναν τρόπο η ρίζα των δεδομένων μας. Έπειτα το κάθε job θα έχει το δικό του bucket στο οποίο θα έχουμε τα εξής dirs:

1. input_files: Εδώ θα αποθηκεύεται ουσιαστικά το αρχείου εισόδου του job
2. output files: Εδώ θα αποθηκεύεται το αρχείο εξόδου του job
3. intermediate files: Εδω αποθηκεύονται τα ενδιάμεσα αρχεία που αξιοποιούνται και παράγονται από τους worker για την εκτέλεση του MapReduce.
    3.1. chunks: Διατηρεί τα chunks του input αρχείου όπως προκύπτουν από το splitting που εκτελεί ο manager.
    3.2. mapper_outputs: Αποθηκεύονται τα αρχεία που εξάγονται από τους map workers.

3.2.1. shuffler_{shuffler_id}: Ο κάθε shuffler έχει ένα bucket από το οποίο θα αναζητήσει τα αρχεία που θα χρησιμοποιήσει ως είσοδο για την ταξινόμηση. Τα αρχεία αυτά φέρουν το όνομα του mapper που τα δημιούργησε. Ο shuffler επομένως αρκεί να βρει το bucket με το ID του στο όνομα.

3.3. shuffler_outputs: Αποθηκεύονται τα αρχεία που παράγουν οι shuffler.

3.3.1. reducer_{reducer_id}: Ο κάθε reducer παράγει το αρχείο που θα χρησιμοποιηθεί ως είσοδος στον reducer με όμοιο ID, οπότε τοποθετεί το αρχείο στο bucket αυτό ώστε ο reducer να χρειάζεται να βρει το bucket με το ID του για την διεργασία του. Το αρχείο φέρει το όνομα του shuffler που το δημιούργησε.

3.3.2. temp: Το bucket αυτό διατηρεί τα sorted αρχεία που προκύπτουν από το πρώτο sort των shuffler. Είναι ουσιαστικά τα αρχεία που προέκυψαν από τα mapper, ταξινομημένα.

3.3.2.1. shuffler_{shuffler_id}: Κάθε shuffler έχει το bucket του για να αποθηκεύσει τα αρχεία του πριν τα ενώσει στο τελικό του output που θα αποθηκεύσει στο 3.3.1.

3.4. reducer_outputs: Περιέχει τα αρχεία που παράγουν οι reduce workers.

Μόλις οι reducer ολοκληρώσουν την διεργασία τους, ο manager θα ενώσει τα αρχεία αυτά σε ένα τελικό merged αρχείο το οποίο και θα δωθεί στον χρήστη.